arXiv:2606.21868v2 [cs.LG] 30 Aug 2026

WiSP: A Working-Set View of Mixture-of-Experts Serving on Extremely Low-Resource Hardware

Jiamu Zhang [1] **Liang Wu** [1] **Mayank Darbari** [1] **Liangjie Hong** [1]

Abstract

Modern local and agentic workloads often need large-model capacity at low concurrency, but run on GPUs that cannot keep a frontier-scale model resident. Mixture-of-Experts (MoE) models are a natural fit because they activate only a small subset of experts per token, but their sparsity saves computation, not residency: the full expert pool still has to be stored, and any expert used by a layer must be in GPU memory when that layer runs. Static layer-level CPU offload makes such models fit, but transfers the expert layer in bulk on every forward pass, losing much of the sparsity advantage. We view low-resource MoE serving as a *working-set* problem on the GPU. Routed expert weights and the KV cache are two memory-demand streams competing for the same limited VRAM. We implement this view in **WiSP** (**W**orking-**S**et **P**aging), a routing-aware expert pager that plugs into an unmodified serving engine and preserves byte-identical outputs. On a real 24 GiB RTX 3090, WiSP achieves up to 2.0× the decode throughput of static offload at the same memory budget when the model does not fit. A natural next step is to predict future experts and prefetch them. We find that this does not help in single-stream decode: the bottleneck is PCIe bandwidth, not prediction quality, so speculative transfers compete with demand transfers instead of hiding them. This shifts the design question from prefetching to allocation: how should one VRAM budget be divided between resident experts and the KV cache? We answer with **MV-WSA** (**M**arginal-**V**alue **W**orking-**S**et **A**llocation), which splits memory by marginal latency benefit per byte while enforcing a KV-admission floor. As a startup configurator, MV-WSA is the only policy we test that stays near-best on both prefill and decode; as a live controller, it resizes both pools while serving and reduces end-to-end time by up to 1.19× over a fixed offline split, without changing model outputs. Our code is publicly available at https://github.com/nokia-applied-research/WiSP.

1 Introduction

Many language-model workloads now run outside large datacenters. Local coding assistants, on-device agents, and shared inference on fractional GPUs all need interactive latency at low concurrency, often on hardware that was not provisioned for frontier-scale models. These workloads still benefit from high-capacity models: an agent may switch between code, planning, tool use, and long-context reasoning within one session, so the deployment wants model capacity even when only one or a few requests are active.

Mixture-of-Experts (MoE) models (Shazeer et al., 2017; Fedus et al., 2022) are an attractive fit for this regime because they decouple capacity from per-token computation. Models such as Qwen3-30B-A3B (Qwen Team, 2025), Mixtral-8×7B (Jiang et al., 2024), and DeepSeek-V3 (DeepSeek-AI, 2024) place much of their parameter count in experts, but a token activates only a small routed subset. In principle, this gives a local or agentic deployment access to a large model's capacity without paying dense-model compute on every token.

The obstacle is that sparsity saves computation, but not residency. A token may use only a few experts, but the full expert pool still has to be stored somewhere, and any expert used by a layer must be in GPU memory when that layer runs. Today's MoEs have tens to hundreds of billions of parameters, so commodity GPUs cannot keep the whole model resident even though only a small fraction is active at any step.

A common way to run these models on small GPUs is offloading: keep most weights in host memory and transfer them over PCIe when the forward pass reaches them. This makes the model fit, and its simplest form is static layer-level offload, such as vLLM's `--cpu-offload-gb` (Kwon et al., 2023). Static offload can serve a 30B MoE model in well under 10 GiB of GPU memory, but it treats an MoE layer much like a dense one: when the layer runs, it transfers the expert weights for the layer even though routing will use only a small subset for the actual computation.

[1]Nokia, Sunnyvale, CA 94085, USA. Correspondence to: Jiamu Zhang <jiamu.zhang@nokia.com>.

Preprint. Under Review.

Several specialized systems improve on this baseline, but they make different tradeoffs. Some keep part of the MoE computation on the CPU, as in ktransformers, which can be fast but relies on Intel AMX (KVCache.AI, 2024), while others target edge deployment by changing the weight format or precision, as in llama.cpp (Gerganov et al., 2023). A third line pipelines, prefetches, or predicts expert transfers to hide communication under computation (Cao et al., 2025; Du et al., 2024; Eliseev & Mazur, 2023). This last line is closest in spirit to WiSP, but its main lever is overlap: it works best when there is enough computation or batching to hide data movement. Our setting is the other end of the spectrum: in low-concurrency interactive serving, each step activates only a small routed subset of experts, but decode exposes little compute behind which to hide PCIe transfers.

The missing piece is therefore not another way to stream the next experts, but a way to decide which experts should remain resident, and how that resident set should share memory with the KV cache.

Serving is a working-set problem. In low-resource MoE serving, the scarce resources are GPU memory and PCIe bandwidth, not arithmetic. This makes the problem look much like the classical *working-set* problem (Denning, 1968; 1970). GPU memory is the small fast store, while host memory is the backing store, and the model generates two streams of memory demand: routed expert weights, determined by the MoE router, and KV-cache blocks, determined by the active tokens. Both have to be resident when used, and both draw from the same VRAM budget.

Under this view, good serving is memory management. The system should keep the experts a workload reuses in GPU memory, leave enough space for the KV blocks of active requests, and avoid admitting more concurrent work than those two working sets can support. This turns expert offload and KV-cache management from two separate knobs into one allocation problem: every byte given to resident experts is a byte taken away from KV, and vice versa. The same abstraction applies to any serving stack with routed experts and an autoregressive KV cache, including MoE LLMs and emerging MoE vision–language models.

The same view also shapes the system design. WiSP changes where weights live and how memory is split, not the computation itself. It leaves the serving engine, kernels, model weights, and numerical precision unchanged, so its outputs are byte-identical to the underlying engine.

WiSP implements this policy as a drop-in plug-in. It reuses the engine's existing expert-map indirection, which maps logical expert ids to physical slots, and makes those slots a small GPU scratch space containing only the resident experts. As the working set changes, WiSP updates the map rather than changing the kernel, model, or precision. This keeps the mechanism architecture-independent for any MoE model the engine can already serve. Section 3 describes the mechanism in detail.

What we find. At matched GPU-memory budgets in the sub-fitting regime, WiSP's routing-aware paging improves decode throughput by up to $2.0\times$ over static offload, measured on a physically constrained 24 GiB RTX 3090 (PCIe 4.0). This is the regime WiSP is designed for: when the model cannot be kept resident, caching only the experts a workload reuses moves far fewer bytes than static layer-level offload. Once enough VRAM is available to keep the model resident, static residency is faster because WiSP's paging indirection becomes unnecessary; we report that crossover explicitly.

We then test the natural next idea: use routing predictions to prefetch experts and hide page faults. In low-concurrency decode, this provides little benefit and can hurt, because there is too little computation behind which to hide a PCIe transfer, and thus decode is bandwidth-bound rather than prediction-bound; speculative transfers mostly compete with demand transfers for the same link. Routing information is therefore more useful as a memory signal—which experts are worth keeping resident—than as a latency signal for prefetching.

This shifts the paper's main question from prefetching to allocation. Expert weights and the KV cache share one VRAM budget: every byte given to resident experts is a byte taken away from KV, and vice versa. We introduce MV-WSA, a marginal-value allocator that splits this budget by equalizing latency benefit per byte, while enforcing a KV-admission floor so the scheduler can still admit the requested concurrency. MV-WSA can be used as a startup configurator, which fixes the split before serving, or as an online controller, which resizes the expert and KV pools while requests are running. In our real-serving allocation test, the configurator is the only policy near-best on both prefill and decode; the online controller further improves end-to-end performance by up to $1.19\times$ over the best offline split at the same VRAM budget, without changing model outputs.

We make four contributions:

- **A working-set formulation of low-resource MoE serving.** We frame routed expert weights and the KV cache as two memory-demand streams competing for the same limited VRAM. This gives a common vocabulary for expert residency, KV allocation, thrashing, and admission, and separates our low-concurrency setting from high-batch datacenter serving (Section 2).
- **A drop-in routing-aware expert pager.** We implement expert paging through the serving engine's ex-

isting expert-map indirection, without changing kernels, model weights, or quantization. On a physically constrained 24 GiB RTX 3090, WiSP improves decode throughput by up to $2.0\times$ over static offload at matched VRAM when the model does not fit (Sections 3, 4).

- **A negative result on prefetching in low-concurrency decode.** We show that routing-based expert prefetching is not the main speed lever in this regime: single-stream decode is PCIe-bandwidth-bound, not prediction-bound, so speculative transfers compete with demand transfers instead of hiding them. The routing signal is more useful for sizing the resident expert set than for reducing latency through prefetching (Section 4).

- **MV-WSA for joint expert–KV allocation.** We introduce a marginal-value allocator that divides VRAM between resident experts and the KV cache under an admission floor. As a startup configurator, it is near-best on both prefill and decode in our real-serving allocation test. As an online controller, it physically resizes both pools during serving and improves end-to-end performance by up to $1.19\times$ over the best offline split at the same VRAM budget, byte-identically (Sections 3, 5).

2 Background: Serving as a Working-Set Problem

2.1 The Working-Set View

A program's *working set* is the set of memory pages it has used recently (Denning, 1968; 1970). If that set stays resident, execution is fast, but if the resident set is too small, the program repeatedly faults on pages it just used and begins to *thrash*. If the system admits more concurrent work than the combined working sets can support, the whole machine thrashes. Low-resource MoE serving has the same shape: each active token needs a routed subset of experts in every MoE layer, and each active request needs its KV blocks. Both must be resident in GPU memory when used. As concurrency, context length, or the routed expert union grows, the two working sets compete for the same limited VRAM.

This view is useful because expert use is sparse, but it is not structureless. A token activates only k of N experts, so keeping every expert resident wastes memory when VRAM is limited. At the same time, a short sequence of decode steps touches much more than a tiny per-token set, so the cache cannot shrink to just the next token's experts. The opportunity is reuse at the workload scale: across bursts and sessions, a workload tends to revisit a stable subset of experts. Caching that routed working set, rather than the whole expert pool, is what makes expert paging viable on a device that cannot keep the full model resident.

The KV cache is the other half of this memory problem. PagedAttention (Kwon et al., 2023) already manages KV as blocks that can be allocated, reused, and spilled. Expert weights, however, are usually handled by a separate mechanism: either kept fully resident or streamed in bulk from the host. The two mechanisms still draw from the same VRAM budget. Every byte used for resident experts is a byte unavailable to KV, and vice versa. Seen this way, low-resource MoE serving is not two independent problems—expert offload and KV management—but one allocation problem over two competing working sets.

2.2 Why the Serving Regime Matters

The best strategy depends on concurrency. In high-batch datacenter serving, a layer may activate many or most experts across the batch. The per-layer working set is then close to the whole expert bank, and there is enough computation to overlap expert transfers with execution. Streaming and prefetching are natural in that setting (Liu et al., 2026; Cao et al., 2025).

WiSP targets the other end of this spectrum: one or a few interactive streams on a GPU that cannot hold the full model. Here each step activates only a small routed subset of experts, and decode exposes little computation behind which to hide PCIe transfers. The useful question is therefore not how to hide the next transfer, but which experts should remain resident across steps and how much VRAM should be left for KV. Section 4 measures this directly.

2.3 Scope and Generality

The expert-paging mechanism applies to any model with routed experts that the serving stack presents as an expert bank. The full working-set allocation problem applies when the model also has an autoregressive KV cache, because resident experts and KV blocks compete for the same memory budget. Models without a KV cache, such as diffusion MoEs or pure recurrent SSM–MoE variants, can still use expert paging to reduce resident memory. They do not exercise the full expert–KV allocation problem, so we treat them as mechanism-generalization cases rather than headline throughput targets. Table 1 summarizes this distinction.

Where WiSP sits among existing systems. Most prior MoE-offloading work targets autoregressive MoE LLMs. The main axis separating these systems is the serving regime. High-batch systems can stream or prefetch experts because they have enough computation to overlap transfers. WiSP targets low-concurrency serving, where overlap is limited and reuse-based caching matters more. It changes residency and allocation policy, not the model computation, so the engine's outputs remain byte-identical.

The closest concurrent system is FluxMoE (Liu et al., 2026).

Table 1. Where the working-set view applies. *Expert stream* = routed experts that can be paged; *KV stream* = an autoregressive key–value cache that co-contends for memory. WiSP's mechanism needs the former; its full theory needs both.

Architecture	MoE variant	Representative	Expert stream	KV stream	WiSP coverage
Dense LLM	× (dense)	Llama-3, Qwen2.5	×	✓	n/a
SSM (pure)	✓	BlackMamba (Anthony et al., 2024)	✓	× (recurrent state)	mechanism only
MoE LLM	✓	Mixtral, Qwen3-MoE, DeepSeek-V3	✓	✓	**full (primary)**
MoE VLM	✓	DeepSeek-VL2, Kimi-VL, Aria	✓	✓	**full (breadth)**
Hybrid SSM–MoE	✓	Jamba (Lieber et al., 2024)	✓	✓(attention)	**full**
MoE diffusion LLM	✓	LLaDA-MoE (Zhu et al., 2025)	✓	× (bidirectional)	mechanism only
MoE image diffusion	✓	DiT-MoE (Fei et al., 2024)	✓	× (latent denoise)	mechanism only

Like WiSP, it trades expert residency against KV capacity, but it uses a stream-and-evict policy tuned for high-batch overlap. WiSP instead uses cache-and-reuse for the regime where there is little compute to hide transfer latency. Beyond autoregressive LLMs, MoE VLMs, hybrid models, and diffusion MoEs use different serving stacks; WiSP attaches to them either as the same serving stack plug-in or through a small wrapper. Appendix D gives the full design-space and serving-stack comparisons (Tables 5, 6).

3 WiSP

WiSP separates low-resource MoE serving into two decisions. First, a *routing-aware expert pager* (§3.2) decides which experts are resident in GPU memory. It uses the serving engine's existing `expert_map` indirection to map routed expert ids onto a small GPU scratch space, so the kernel, weights, precision, and outputs are unchanged. Second, *MV-WSA* (Marginal-Value Working-Set Allocation, §3.3–§3.5) decides how much of the available VRAM should go to that expert scratch versus the KV cache. The pager manages residency *within* the expert pool; MV-WSA moves the boundary *between* the expert and KV pools. We use MV-WSA in two modes: offline, as a startup configurator that fixes the split before serving, and online, as a controller that resizes both pools while requests are running. Figure 1 summarizes the design.

3.1 Setting and Notation

The model has L MoE layers with N experts each, and the router selects $k \ll N$ experts per token per layer. At step t, layer ℓ selects $R_\ell^{(t)} \subseteq \{1, \dots, N\}$, and every expert $e \in R_\ell^{(t)}$ must be resident when that layer executes. After fixed GPU costs are reserved—non-expert weights, activations, CUDA context, and other runtime state—the device has a pageable budget B shared by resident experts and the KV cache:

$$B = B_{\text{exp}} + B_{\text{kv}}, \qquad f \triangleq B_{\text{exp}}/B \in (0,1). \quad (1)$$

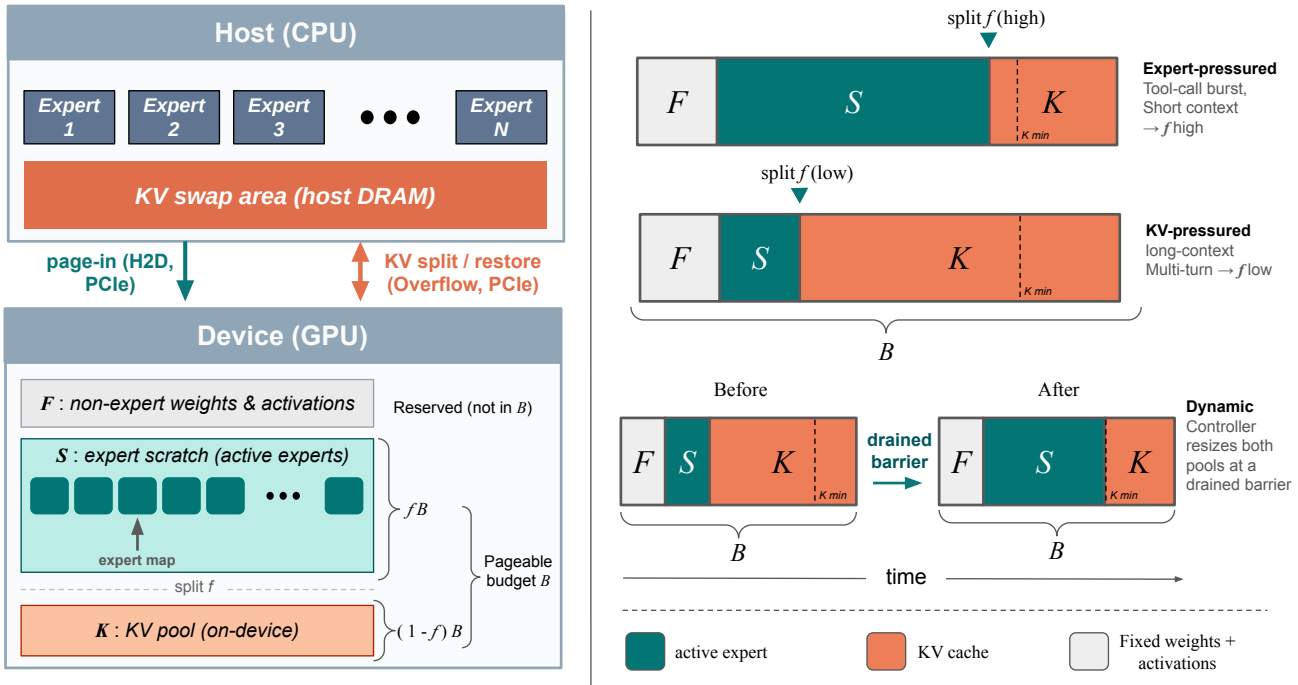
The split f is MV-WSA's control variable. If one expert occupies M_e bytes, then increasing the per-layer resident cap by one expert costs $a \triangleq LM_e$ bytes across all MoE layers, so the expert budget supports $C = B_{\text{exp}}/a$ resident experts per layer. The KV budget supports $\kappa = B_{\text{kv}}/b$ KV blocks, where b is the GPU memory per KV block summed over all layers. At a fixed budget B, WiSP's allocation problem is to choose where this split should land.

3.2 The Expert Pager

The allocator can only trade expert memory against KV memory if the expert side is itself pageable. WiSP provides this substrate with the engine's existing expert-map indirection, without changing the fused-MoE kernel, model weights, or precision.

Expert-map indirection. Modern fused-MoE kernels accept an `expert_map` π that maps logical expert ids to physical slots. WiSP makes those physical slots a small GPU scratch space with capacity C per layer. The full expert weights remain in host memory; at each layer, WiSP ensures that the routed experts are present in scratch, updates π, and dispatches the unmodified kernel. A page fault occurs when a routed expert is not currently resident, in which case WiSP copies it from the host master and evicts an older resident expert using LRU. All dynamic behavior is in the map and the scratch contents, not in the computation.

Per-step protocol. Algorithm 1 shows the per-layer decode step. Given the routed expert set R, WiSP copies only the missing experts, assigns them scratch slots, refreshes the LRU state for the experts used by this step, and calls the stock fused-MoE kernel with the updated expert map. The key invariant is exactness. When the kernel runs, every routed expert id maps to a scratch slot containing the same bytes the full expert tensor would have supplied. The kernel is unchanged, so WiSP's output matches the unpaged engine up to the engine's own floating-point reproducibility. For large prefill batches whose expert union exceeds C, WiSP groups tokens so that each group fits the scratch and reassembles the outputs in token order; this path is an implementation detail and does not trigger in our single-stream decode measurements.



(a) System Layout

(b) Runtime regimes: MV-WSA moves the split

Figure 1. WiSP at a glance. **(a) System layout.** The full expert weights stay pinned on the host; WiSP *pages in* only the routed experts to a small GPU scratch S over PCIe and runs the unmodified fused-MoE kernel through the `expert_map` π, so output is byte-identical. The pageable budget $B = B_{\text{exp}} + B_{\text{kv}}$—the expert scratch S plus the KV pool K, *excluding* the fixed non-expert slab F—is divided by the split f; KV that overflows the on-device pool spills to host. **(b) Runtime regimes.** The optimal split moves with the workload—expert-heavy for tool-call bursts, KV-heavy for long contexts—and MV-WSA tracks it, in the dynamic case by physically resizing both pools at a drained barrier, byte-identically.

Algorithm 1 WISPLAYER.STEP(R) at one MoE layer

Input: needed experts R; cap C; maps π, π^{-1}; LRU ticks τ; clock c; CPU master W^{cpu}; GPU scratch S; device map π_{dev}
$M \leftarrow \{e \in R : \pi(e) = \perp\}$ ▷ faults
$U \leftarrow \{s : \pi^{-1}(s) \notin R\}$ in increasing τ ▷ LRU victims
ASSIGN each $e \in M$ a victim slot s_e from U
for each $e \in M$ with slot s_e **do**
  $S[s_e] \leftarrow$ COPY_ASYNC$(W^{\text{cpu}}[e])$
  $\pi(e) \leftarrow s_e$; $\pi^{-1}(s_e) \leftarrow e$; $\pi_{\text{dev}}[e] \leftarrow s_e$
end for
$c \leftarrow c + 1$; **for** $e \in R$ **do** $\tau[\pi(e)] \leftarrow c$
return FUSEDMOE(hidden, $W{=}S$, `expert_map`$=\pi_{\text{dev}}$)

Reuse-distance miss curves. WiSP uses LRU because the low-concurrency decode regime is reuse-limited rather than overlap-limited: each step has too little computation to hide a PCIe transfer, so the useful policy is to keep the experts a workload comes back to. LRU also gives a simple way to estimate the value of more memory. From a reference trace, the reuse-distance histogram determines the miss curve $m(C)$: how many references would miss at resident cap C. We use this miss-curve view for the expert stream, and use the analogous block-level view for KV, with the admission floor below capturing the scheduler cliff that a smooth miss curve misses.

3.3 The Expert↔KV Split

A split f gives $C = fB/a$ expert slots per layer and $\kappa = (1-f)B/b$ KV blocks. The allocator asks a simple question: if we move one byte from KV to experts, do we remove more latency than if we leave that byte in KV?

Let $m_{\text{exp}}(C)$ be the expert miss count at cap C, and let $m_{\text{kv}}(\kappa)$ be the KV miss or recompute cost induced by a KV pool of κ blocks. Expert misses and KV misses have different costs. An expert miss transfers weights over PCIe, with cost $c_{\text{exp}} = M_e t_{\text{byte}}$. A KV miss costs recomputation, with cost $c_{\text{kv}} = \beta b_{\text{tok}}$. The latency model at split f is

$$T(f) \;=\; c_{\text{exp}}\, m_{\text{exp}}(C) \;+\; c_{\text{kv}}\, m_{\text{kv}}(\kappa). \qquad (2)$$

The best split depends on the workload. Short tool-call bursts have little KV pressure and benefit from keeping more reused experts resident, so the optimum shifts expert-heavy. Long-context or multi-turn workloads put more pressure on KV, so the optimum shifts back toward KV. At an interior

optimum, the split is equimarginal: one more byte should buy the same latency reduction in either pool,

$$c_{\text{exp}}\, m'_{\text{exp}}(C)/a \;=\; c_{\text{kv}}\, m'_{\text{kv}}(\kappa)/b. \tag{3}$$

MV-WSA follows this principle: move bytes toward whichever pool currently removes more latency per byte.

The admission floor. The smooth cost in Eq. (2) misses one hard constraint. A continuous-batching scheduler must have enough KV capacity to admit the active requests. If the GPU KV pool is too small, the scheduler may preempt and re-prefill repeatedly, or the engine may refuse to initialize. This is a cliff, not a gradual miss penalty. We therefore enforce

$$\kappa \;\geq\; \kappa_{\min} \;=\; W \cdot \lceil \ell_{\text{ctx}}/b_{\text{tok}} \rceil, \tag{4}$$

where W is the served concurrency and ℓ_{ctx} is the per-session context length. The floor is slack when KV is the pressured pool. It becomes binding when the unconstrained optimum pushes the split toward experts: $\kappa_{\min}$ is what keeps an expert-heavy split admissible. This is the difference between MV-WSA and a KV-blind expert-proportional split, which we measure as a startup failure in Section 5.3.

3.4 MV-WSA: Configurator and Online Controller

MV-WSA does not need the full miss curves in advance. It builds them from decayed reuse-distance histograms while the workload runs, then moves the split toward the equimarginal point in Eq. (3). A small hysteresis band prevents the split from chasing noise. We use the same allocation principle in two modes.

In *offline* mode, MV-WSA runs on a short representative trace and returns a fixed split $\hat{f}$ used at startup. Given a budget B, it emits the two engine knobs that realize this split: the per-layer expert cap and the GPU KV pool, subject to the admission floor,

$$C = \min\big(\lfloor \hat{f}B/a \rfloor,\, \lfloor (B - \kappa_{\min} b)/a \rfloor\big), \qquad B_{\text{kv}} = B - Ca. \tag{5}$$

All other memory settings are held fixed, so comparisons are iso-VRAM: the arms differ only in where the same bytes go.

In *online* mode, MV-WSA updates the split while serving. A single startup split is only the best fixed compromise over a trace, but an agentic session can move between regimes: long-context turns may need KV, while short tool-call bursts may leave KV underused and benefit from more resident experts.

3.5 Dynamic Dual-Resize

The live controller implements the online case using a simple observation from the engine: once the KV pool covers the current working set, stays above $\kappa_{\min}$, and has a small amount of headroom, extra KV blocks have near-zero marginal value. The controller therefore measures the peak KV occupancy at a drained barrier, keeps KV just above that peak, and gives the remaining bytes to the expert scratch. In this regime, the equimarginal rule becomes simple: once KV has enough room, the next byte is worth more on the expert side. Appendix B gives the full control law.

Applying the new split inside a running engine requires two resize operations: resizing the per-layer expert scratch and resizing the vLLM KV-block pool. Both happen only at a drained barrier, when no request owns a block being moved. The controller frees the shrinking side before growing the other, so the transient footprint never exceeds the fixed budget B. The result is still an iso-VRAM comparison, in which the controller only moves bytes from one pool to the other. Because it moves stored tensors rather than changing computation, greedy outputs are preserved. If a resize fails, the engine keeps the previous valid split.

3.6 Implementation

WiSP is implemented as a drop-in plug-in over an unmodified vLLM. Expert masters live in host memory, and each MoE layer owns a GPU scratch whose size is set by the expert cap C. The static configurator realizes a split by setting the expert cap and the on-device KV pool; the online controller adds resize operations for both pools and applies them only at drained barriers. Appendix C gives the engineering details.

4 The Central Finding: Prediction Is a Memory Lever, Not a Speed Lever

The working-set view suggests a natural optimization: if we can predict which experts a token will need, we should prefetch them and hide page faults. This is not a strawman. Routing is partly predictable because later-layer routing depends on hidden states produced by earlier experts, and our co-activation predictor reaches nontrivial next-expert precision (Appendix E). The question is whether that prediction turns into decode speed.

In the low-concurrency regime, it does not. Figure 2 tests speculative expert prefetching on Qwen3-30B-A3B on a real 24 GiB RTX 3090 (PCIe 4.0). Turning on prefetch lowers decode throughput by 38–60% at constrained caps and raises time-to-first-token by 1.6–$1.9\times$. A second test compares a topic-coherent session with a deliberately diverse one; despite having more routing locality, the coherent session reaches no higher steady-state throughput (6.47 vs. 6.82 tok/s). There is no warm-up benefit to harvest.

The reason is bandwidth. At batch-1 decode, each layer

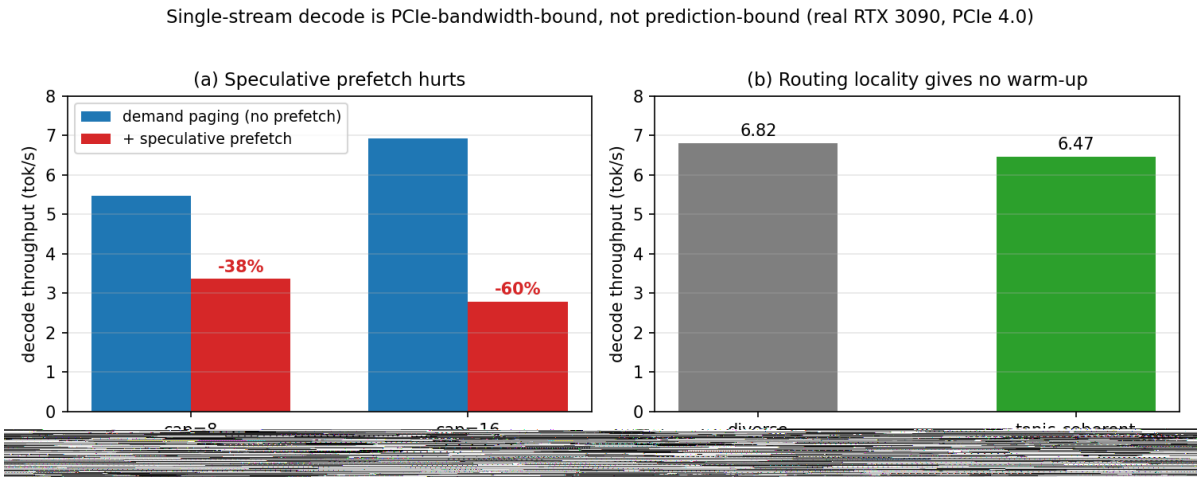


Figure 2. Prediction does not buy decode speed at single-stream, measured on a real 24 GiB RTX 3090 (PCIe 4.0). (a) Turning on speculative prefetch with the online co-activation predictor *lowers* decode throughput by 38–60% at constrained caps (cap 8: 5.47 → 3.37; cap 16: 6.94 → 2.79 tok/s) and raises time-to-first-token by 1.6–1.9× (Qwen3-30B-A3B). (b) A topic-coherent (code) session reaches no higher steady-state throughput than a deliberately diverse one (6.47 vs. 6.82 tok/s), so there is no routing-locality warm-up to harvest. The binding constraint is PCIe bandwidth, not prediction quality.

computes a single token, so the MoE GEMM is too small to hide a side-stream PCIe transfer. Speculative copies therefore compete with demand copies for the same link, and at tight caps they can evict experts that the next step still needs. The measured traffic makes the limit concrete: on Qwen3-30B-A3B, only 44–48% of each layer's top-8 experts carry over between adjacent decode steps. Even a perfect last-step cache must still stream about 1.8 GiB of expert weights per token. On our RTX 3090, with about 25 GB/s host-to-device bandwidth, that mandatory traffic alone caps decode at roughly 13–16 tok/s. The pipe is already full; prediction can reshuffle traffic, but it cannot widen the pipe.

This result changes how we use routing information. In this regime, routing is not a latency signal for prefetching; it is a memory signal for deciding how large the resident expert set should be. Once expert paging frees part of the model from GPU residency, the next question is how to spend that freed memory: keep more experts resident, or give the bytes to the KV cache? That allocation problem is what MV-WSA solves (§3.3–§3.5) and what we evaluate in §5.3.

5 Evaluation

The evaluation follows the paper's argument. First, WiSP should not change the model's computation, so we check output preservation (Section 5.1). Second, if expert residency is a working-set problem, routing-aware paging should move less data than static layer-level offload at the same GPU budget; we test this on a physically constrained 24 GiB RTX 3090 (Section 5.2). Third, because expert memory and KV memory share one store, we evaluate whether MV-WSA chooses a better split than fixed or memory-blind policies, both as a startup configurator and as a live resize controller (Section 5.3).

Experimental setup and scope. The headline iso-VRAM throughput results for Qwen3-30B-A3B and Kimi-VL-A3B (Figure 3, Table 2) are measured on a physically constrained consumer GPU: a single 24 GiB NVIDIA RTX 3090 on a real PCIe 4.0 x16 link (measured host-to-device bandwidth ≈25 GB/s), vLLM 0.11.2, eager mode, temperature 0. Both models exceed 24 GiB in BF16 (Qwen3 ≈57 GiB, Kimi-VL ≈31 GiB), so paging is forced by the hardware rather than emulated. The H100 results serve different purposes: MiniMax-M2 and Jamba are capability checks on models that exceed one 94 GiB H100, while the OLMoE MV-WSA startup-configurator test isolates the allocation policy under a controlled 8 GiB budget. The diffusion experiments use a standalone PyTorch pager, so we report correctness and resident-memory behavior there, not serving latency.

5.1 Paging Is Output-Preserving

At temperature 0, WiSP matches the unmodified engine for every cap we test, and this is the expected consequence of property P1: the fused-MoE kernel is unchanged, and every routed expert is read from byte-equal weights. We also check the mechanism on diffusion models, where the comparison is stricter because we can inspect the whole generation trajectory: DiT-MoE latents match with maximum absolute difference zero across the cap sweep, and LLaDA-MoE completions are token-identical (Section 5.4).

5.2 Iso-VRAM Decode Throughput versus Static Offload

Table 2 summarizes the main system cases. Qwen3 and Kimi-VL are matched-budget throughput comparisons on the 24 GiB edge GPU. Jamba and MiniMax-M2 are capability checks on models whose native serving paths fail or become impractical under the stated memory constraints. The diffusion mechanism-only cases have no serving-throughput baseline and are reported in Appendix F.

Figure 3 is the main systems result on real edge hardware. On the 24 GiB RTX 3090, both WiSP and the static-offload baseline run under the same GPU budget, but they spend the bytes differently. Static offload streams the expert layer in bulk, so its decode throughput stays near 4–5 tok/s. WiSP keeps a routed expert working set resident, so throughput rises as the cap grows: 6.0, 7.6, and 9.9 tok/s for cap 8, 16, and 32. The result is a $\approx$1.8–2.0$\times$ speedup in the sub-fitting range this card can actually exercise. A 24 GiB card cannot reach the full-residency crossover; we only claim the regime where the model does not fit.

Kimi-VL gives a second check on the same mechanism in a different model family. It is a MoE vision–language model with a dense vision tower and a routed MoE language backbone, yet WiSP uses the same vLLM plug-in without model-specific code. At matched budgets, it improves decode throughput by 1.58–1.79$\times$ over static offload (Table 2) while preserving outputs on both text and image prompts (Section 5.4).

Scaling to MiniMax-M2: a capability check. We also run WiSP on the 229B-parameter FP8 MiniMax-M2 on one 94 GiB H100. This is a capability check, not a throughput claim. WiSP serves the model with coherent output while keeping only a 7–14 GiB expert scratch on the GPU. Stock vLLM can also serve the model on this card with enough static offload, and is faster, but lowering its GPU footprint requires more offloaded and pinned/staged host memory until host RAM becomes the bottleneck. WiSP's point here is different: the full expert set lives once in host memory, while the GPU holds only the resident scratch. A clean iso-GPU-VRAM comparison where this tradeoff pays off requires a physically smaller card, which we leave to future hardware work.

The throughput result is only the expert side of the trade. The working-set view says that the expert cache and the KV cache share one store: shrinking the resident expert cap leaves more GPU memory for on-device KV, while growing the expert cap reduces the KV pool available to the scheduler. Push the expert cap too high and admission can fail because the engine has no usable KV pool. Table 4 measures the reverse move online, reclaiming idle KV capacity and giving those bytes back to resident experts. This is the allocation problem MV-WSA is designed to solve.

5.3 Allocating the Budget: MV-WSA

MV-WSA chooses the split between resident experts and on-device KV. We evaluate it in two settings. First, as a static startup configurator, it chooses a fixed split before serving and we compare that split against fixed and KV-blind policies on real `vllm serve` (Table 3). Second, as a live controller, it physically moves bytes between the expert scratch and the KV pool while requests are running (Table 4).

Static configurator, real serving. We run one `vllm serve` instance per arm and replay `os` sessions from AgentInstruct (Zeng et al., 2024), the agent-trajectory dataset whose `os`/`db`/`webshop` splits are collected on AgentBench environments (Liu et al., 2024a), at concurrency 4. Each session is prepended with a distinct $\sim$1.2–1.5k-token pseudo-document, so concurrent requests hold separate KV state rather than sharing a cached prefix. All arms use the same total GPU budget and host CPU-swap; only the expert–KV split changes.

We compare four policies: **MV-WSA**, which uses the configurator split $\hat{f}$ subject to the admission floor; **50–50**, an even split of the pageable pool; **KV-blind prop.**[1], an expert-working-set-proportional split that ignores KV demand; and **naive offload**, vanilla `--cpu-offload-gb` with WiSP off. Table 3 runs this on OLMoE-1B-7B at an 8 GiB budget, a KV-pressured regime where wasting KV capacity shows up immediately in TTFT.

Table 3 shows the two failure modes. **KV-blind prop.** gives the most memory to experts and therefore has the fastest decode, but it leaves only 0.5 GiB for KV; TTFT rises to 19.8 s and turn wall reaches 22.0 s. **50–50** leaves more KV, but its smaller expert cap slows decode to 29.2 tok/s. **Naive offload** streams every expert and pays on both axes. **MV-WSA** balances the two pools: it matches the best TTFT, keeps decode well above 50–50, and has the lowest median turn wall.

The admission floor is what makes the right split feasible. The KV-blind prior in Table 3 shows the soft version of KV starvation: the engine runs, but TTFT collapses. The hard version is a startup failure. If an allocator pushes the split too far toward experts, the remaining KV pool may be too small to admit even one requested context, and the engine can refuse to initialize. The admission floor in Eq. (4) prevents this by clamping the split to the most expert-heavy allocation that still satisfies the scheduler's KV requirement.

[1]This is a deliberately KV-blind prior, not FluxMoE (Liu et al., 2026), which targets high-batch stream-and-evict overlap.

Table 2. Main results across the full-system cases. Where a throughput comparison is possible we match GPU-memory budgets: *naive offload* is vLLM's static `--cpu-offload-gb` (offloaded weight bytes, in GiB) and *WiSP paging* keeps a cap-sized resident expert scratch (cap, in experts/layer); both columns report decode *tok/s* at that budget. EDGE = measured on a physically constrained 24 GiB RTX 3090. REAL/CAP. = the model exceeds one 94 GiB H100 or hits an unsupported native path, so the native baseline does not run (×); these rows are capability checks rather than throughput claims and have no speedup ("—").

		naive offload		WiSP paging		
Setting	GPU budget	GiB	tok/s	cap	tok/s	Speedup
MoE LLM — Qwen3-30B-A3B (BF16), 128 experts, top-8, on a 24 GiB RTX 3090 / PCIe 4.0 (cf. Figure 3)						
EDGE	≈20 GiB	50	4.17	16	7.62	1.83×
EDGE	≈22 GiB	44	4.93	32	**9.93**	**2.01**×
MoE VLM — Kimi-VL-A3B (BF16), 64 experts, top-8, on a 24 GiB RTX 3090 / PCIe 4.0						
EDGE	≈14.7 GiB	20	5.00	16	8.97	1.79×
EDGE	≈8.8 GiB	24	4.23	8	6.69	1.58×
Hybrid SSM–MoE — Jamba-v0.1 (52B, 103 GiB BF16), 16 experts, top-2						
REAL	94 GiB H100	full residency: × OOM (103>94)		4	5.73	—
REAL	94 GiB H100	`cpu-offload-gb` (any): × assertion		8	9.80	—
MoE LLM — MiniMax-M2 (229B FP8), 256 experts, top-8						
CAP.	94 GiB H100	offload ≥180: × host OOM (256 GiB cgroup)		32	3.70 (14 GiB exp.)	—

Table 3. MV-WSA startup configurator (*static* split, fixed for the run) on real `vllm serve`, iso-VRAM, AgentInstruct (Zeng et al., 2024) `os`, concurrency 4, OLMoE-1B-7B at an 8 GiB budget. Every arm gets the same total GPU budget and host CPU-swap; only the expert–KV split changes. *Turn wall* is median wall-clock time per turn (prefill+decode), lower is better.

Arm (split)	cap	KV (GiB)	TTFT med	decode	turn wall (s)
MV-WSA	21	2.6	**2.9**	40.6	**6.3**
50–50	18	3.2	2.9	29.2	7.0
KV-blind prop.	35	0.5	19.8	**56.2**	22.0
naive offload	—	—	21.3	30.7	25.5

Live dual-resize: reclaiming idle KV. The closed-form controller of §3.5 runs in the in-process engine and resizes both pools at drained barriers. On the 24 GiB RTX 3090, we replay two Qwen3 AgentInstruct traces (`os`, `db`) at concurrency 4. In both traces, the fixed offline split reserves more KV than the workload actually uses: the KV pool peaks at only ≈44% of its allocation each round. The controller observes this after the first drained round and moves the unused KV budget to resident experts (cap 32 → 35, KV 2.79 → 1.53 GiB), then holds that split with zero preemptions.

Table 4 shows the effect. Dynamic resizing reduces end-to-end time by 1.07–1.19× over the fixed offline split at the same GPU budget, and improves TTFT by 1.11–1.27×. Decode improves on `db` and is essentially unchanged on `os`. A separate resize-cycle check confirms property (D2): outputs remain byte-identical across a shrink-KV/grow-experts and grow-KV/shrink-experts cycle. The gain is regime-dependent: when the workload is already KV-pressured, as in Table 3, the static configurator is already near the right split and there is little idle KV to reclaim.

Table 4. Live MV-WSA dual-resize (*dynamic*: both pools resized *while* serving) vs. the fixed offline split, on a real 24 GiB RTX 3090, Qwen3-30B-A3B, AgentInstruct (Zeng et al., 2024) `os` and `db`, concurrency 4, 36 timed turns. Both arms replay the byte-identical trace; the only difference is the policy. Lower is better except decode. The dynamic arm reclaims idle KV into experts in one move (cap 32 → 35, KV 2.79 → 1.53 GiB) and reduces end-to-end time on both traces.

Trace	Arm	cap	KV pool	TTFT med (s)	decode (tok/s)	e2e (s)
`os`	fixed	32	2.79 GiB	89.31	3.17	1237.1
	dynamic	32→**35**	2.79→**1.53** GiB	**80.74**	3.14	**1155.0**
	dyn/fix	—	—	1.11×	0.99×	**1.07**×
`db`	fixed	32	2.79 GiB	95.46	3.02	1355.4
	dynamic	32→**35**	2.79→**1.53** GiB	**75.05**	**3.36**	**1138.4**
	dyn/fix	—	—	1.27×	1.11×	**1.19**×

5.4 Generality across Architectures and Modalities

The expert-paging mechanism needs only routed experts, so it carries beyond text-only MoE LLMs. On **Kimi-VL-A3B**, the same vLLM plug-in pages the MoE language backbone with no model-specific code, preserves outputs on both text and image prompts, and improves decode throughput at matched budgets (Section 5.2). On the hybrid **Jamba-v0.1** (52B; Mamba+attention+MoE), WiSP serves the model on one 94 GiB GPU in a regime where stock vLLM cannot: full residency exceeds VRAM, and the CPU-offload path is unsupported for Jamba's recurrent state. This is a capability claim scoped to the hybrid model. For MoE diffusion models without an autoregressive KV cache, the full expert–KV allocation problem does not apply, but the pager still does. A roughly 150-line PyTorch wrapper pages **DiT-MoE** and

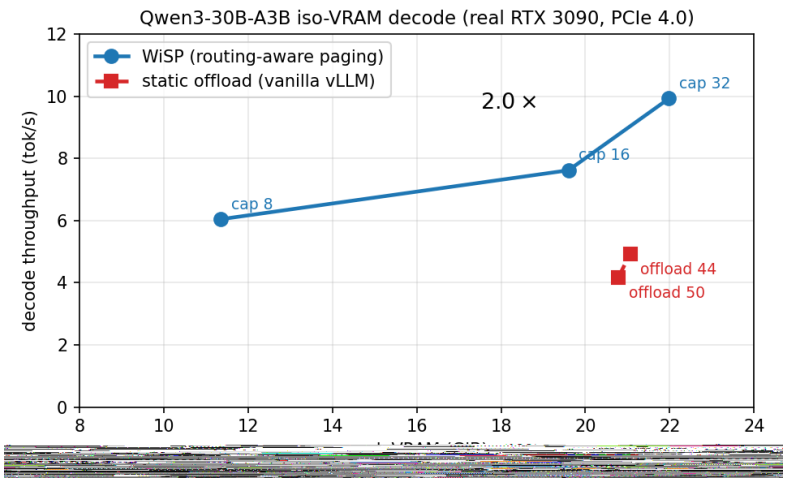


Figure 3. Iso-VRAM decode throughput on Qwen3-30B-A3B, measured on a real 24 GiB RTX 3090 (PCIe 4.0). When the model does not fit, routing-aware paging (WiSP) moves fewer bytes per step than static layer-level offload. At a matched ≈22 GiB budget, WiSP decodes ≈2.0× faster (cap 32: 9.93 vs. offload 44: 4.93 tok/s). Within the range this card can hold, increasing the expert cap improves WiSP throughput (cap 8/16/32 → 6.0/7.6/9.9 tok/s), while the routing-blind baseline remains bandwidth-limited. A 24 GiB card cannot reach the full-footprint budget where the model fits and static residency wins; WiSP targets the sub-fitting regime, not a universal speedup.

LLaDA-MoE-7B; generation remains bit-/token-identical to full residency, while resident expert memory drops by up to $4\times$. These models also expose different working-set structures: DiT has a fully activated per-forward stream with little cross-step reuse, while LLaDA has strong temporal locality across denoising steps. Appendix F reports the full mechanism-generalization results (Figures 4, 5).

6 Conclusion

Serving an MoE on a GPU that cannot keep it resident is a working-set problem: routed expert weights and the KV cache are two memory-demand streams competing for one small store. WiSP realizes this view for the low-concurrency regime. It pages routed experts through a drop-in, byte-identical vLLM plug-in, improves decode throughput by up to $2.0\times$ over static offload at matched VRAM on a real 24 GiB GPU, and carries the same expert-paging mechanism to a MoE VLM, a 52B hybrid model whose stock vLLM path cannot serve it on one GPU, and two diffusion MoEs as mechanism-generalization checks.

The main lesson is that in single-stream, memory-bound decode, routing prediction is not the speed lever. There is too little compute to hide PCIe transfers, so the routing signal is more useful as a memory signal: it tells us which expert working set is worth keeping resident. That makes allocation the central problem. MV-WSA splits one VRAM budget between resident experts and KV under an admission floor, and the live controller shows that this split can be changed while serving without changing model outputs. Porting the full estimator into the live serve loop, broadening the dynamic controller beyond the current proof of concept, and validating the MiniMax regime on physically smaller GPUs are the next steps.

References

Anthony, Q., Tokpanov, Y., Glorioso, P., and Millidge, B. BlackMamba: Mixture of experts for state-space models. *arXiv preprint arXiv:2402.01771*, 2024.

Cao, S., Liu, S., Griggs, T., Schafhalter, P., Liu, X., Sheng, Y., Gonzalez, J. E., Zaharia, M., and Stoica, I. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Denning, P. J. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

Denning, P. J. Virtual memory. *ACM Computing Surveys*, 2 (3):153–189, 1970.

Du, Z. et al. SiDA: Sparsity-inspired data-aware serving for efficient and scalable large Mixture-of-Experts models. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Eliseev, A. and Mazur, D. Fast inference of Mixture-of-Experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research (JMLR)*, 23:1–39, 2022.

Fei, Z., Fan, M., Yu, C., Li, D., and Huang, J. Scaling diffusion transformers to 16 billion parameters (DiT-MoE). *arXiv preprint arXiv:2407.11633*, 2024.

Gerganov, G. et al. llama.cpp. https://github.com/ggml-org/llama.cpp, 2023.

Huang, Y., Fang, Z., Luo, W., Wu, R., Chen, W., and Zheng, Z. DyMoE: Dynamic expert orchestration with mixed-precision quantization for efficient MoE inference on edge. *arXiv preprint arXiv:2603.19172*, 2026.

Hwang, R., Wei, J., Cao, S., Hwang, C., Tang, X., Cao, T., and Yang, M. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.

Jiang, A. Q. et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Kimi Team. Kimi-VL technical report. *arXiv preprint arXiv:2504.07491*, 2025.

KVCache.AI. KTransformers: A flexible framework for cutting-edge LLM inference optimizations. https://github.com/kvcache-ai/ktransformers, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

Li, D. et al. Aria: An open multimodal native mixture-of-experts model. *arXiv preprint arXiv:2410.05993*, 2024.

Lieber, O. et al. Jamba: A hybrid transformer-Mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

Liu, Q., He, C. Y., Jiang, H., Wang, Z., Zhao, A., and Lee, P. P. C. FluxMoE: Decoupling expert residency for high-performance MoE serving. *arXiv preprint arXiv:2604.02715*, 2026.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., and Tang, J. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024a. URL https://openreview.net/forum?id=zAdUB0aCTQ.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024b.

Nie, S. et al. Large language diffusion models (LLaDA). *arXiv preprint arXiv:2502.09992*, 2025.

Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated Mixture-of-Experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Wu, Z. et al. DeepSeek-VL2: Mixture-of-experts vision-language models for advanced multimodal understanding. *arXiv preprint arXiv:2412.10302*, 2024.

Xue, L., Fu, Y., Lu, Z., Sun, C., Mai, L., and Marina, M. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.

Yang, S., Fan, X., Pan, M., Xi, H., Wang, Z., Sun, S., Keutzer, K., Han, S., Zaharia, M., Xu, C., and Stoica, I. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.

Yu, H., Cui, X., Zhang, H., Wang, H., and Wang, H. Taming latency-memory trade-off in MoE-based LLM serving via fine-grained expert offloading. In *European Conference on Computer Systems (EuroSys)*, 2026.

Zeng, A., Liu, M., Lu, R., Wang, B., Liu, X., Dong, Y., and Tang, J. AgentTuning: Enabling generalized agent abilities for LLMs. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3053–3077, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.181. URL https://aclanthology.org/2024.findings-acl.181/.

Zheng, L. et al. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Zhong, S., Sun, Y., Liang, L., Wang, R., Huang, R., and Li, M. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *ACM/IEEE Design Automation Conference (DAC)*, 2025.

Zhu, F., You, Z., Xing, Y., Huang, Z., et al. LLaDA-MoE: A sparse MoE diffusion language model. *arXiv preprint arXiv:2509.24389*, 2025.

A Related Work

WiSP's conceptual basis is Denning's working-set model and thrashing theory (Denning, 1968; 1970); our contribution is to recognize low-resource MoE serving as a direct instance and use that vocabulary to reason about expert residency, KV capacity, and admission together. On the systems side, PagedAttention in vLLM (Kwon et al., 2023) and RadixAttention in SGLang (Zheng et al., 2024) brought a paging and prefix-sharing discipline to the KV reference stream; we bring the same discipline to the expert stream and treat the two as co-contending working sets over one store.

A line of batch-oriented systems serves MoEs on memory-constrained GPUs by overlapping transfer with compute: MoE-Lightning (Cao et al., 2025) pipelines paged weights and reports up to $10.3\times$ on a T4, SiDA-MoE (Du et al., 2024) uses an offline hash predictor of active experts (approximate, $\sim 1\%$ accuracy drop), and Mixtral-offloading (Eliseev & Mazur, 2023) streams experts for a single stream. These target high throughput at large batch, where there is ample compute to hide transfers. A parallel line predicts or caches experts to cut miss latency rather than stream blindly: MoE-Infinity (Xue et al., 2024) keeps a sparsity-aware expert cache driven by activation tracking on personal machines; Pre-gated MoE (Hwang et al., 2024) co-designs the model to pre-compute the next layer's experts one step ahead; HybriMoE (Zhong et al., 2025) splits expert work across CPU and GPU with an impact-driven prefetch; FineMoE (Yu et al., 2026) retrieves per-request expert maps by semantic and trajectory similarity to guide prefetching; and DyMoE (Huang et al., 2026) assigns experts mixed precision (down to skipping) to shrink edge transfers. Most share WiSP's premise that expert residency is the bottleneck, but spend the routing signal on prefetching or overlap for speed—which Section 4 shows is a poor fit for single-stream decode, where no compute hides a transfer. WiSP instead spends the signal on sizing and treats experts and KV as one budget. The closest concurrent work is FluxMoE (Liu et al., 2026), which, like WiSP, builds on vLLM and reclaims expert VRAM for the KV cache, but adopts a stream-and-evict policy—paging an expert in just before its layer and evicting it immediately, pipelined with layer compute—to maximize high-batch throughput. WiSP differs in four ways: it targets the low-concurrency, cache-and-reuse operating point, where Section 4 shows there is no compute to overlap; it ships as a pure plug-in over the unmodified engine with no virtual-memory machinery; rather than a fixed, overlap-tuned residency plan it *allocates* the expert↔KV split by marginal value (MV-WSA), online and subject to an admission floor, and physically resizes both pools while serving; and it contributes the working-set framing, the measurement of when prediction does and does not pay off, and the cross-architecture breadth. The systems are

complementary across the concurrency axis.

Concurrent with this work, FreeToken (Yang et al., 2026) co-designs a full edge-native MoE serving stack—pipelined full-layer expert loading for prefill, semantic-aware state caching for agentic context edits, and an elastic expert/KV memory mechanism—and, at decode, serves residual expert misses through a second channel: executing missing experts in place on the CPU, splitting each layer's misses between PCIe fill and CPU compute in proportion to profiled bandwidths. This adds a degree of freedom that allocation alone cannot supply under the single-stream PCIe ceiling of Section 4: where WiSP can only fetch a missing expert, FreeToken may also compute it where it resides. The two designs are complementary along the allocation axis: FreeToken provides a runtime mechanism for reconfiguring the expert/KV split but does not specify a policy for choosing or adapting that split, which is the question MV-WSA answers, and extending MV-WSA's marginal-value rule to price a second service channel (fetch versus compute) is a natural next step.

Prior work on MoE routing prediction and speculative loading motivates the predictor we build in Section 4; we evaluate it directly and report a negative speed result for the low-concurrency regime, which clarifies that the signal is better spent on memory than latency. KV-cache compression methods such as KIVI (Liu et al., 2024b) reduce the other side of the joint working set and compose with WiSP rather than competing with it. Finally, WiSP is evaluated across MoE LLMs (Shazeer et al., 2017; Fedus et al., 2022; Jiang et al., 2024; DeepSeek-AI, 2024; Qwen Team, 2025), MoE VLMs (Wu et al., 2024; Kimi Team, 2025; Li et al., 2024), a hybrid SSM-MoE (Lieber et al., 2024; Anthony et al., 2024), and MoE diffusion models for images (Peebles & Xie, 2023; Fei et al., 2024) and language (Nie et al., 2025; Zhu et al., 2025); we use these only as test subjects and change none of them.

B Dynamic Dual-Resize Control Law

Recall $a = LM_e$. At a drained barrier the conserved budget is measured directly, $B = \kappa b + Ca$, and the target sizes $\tilde{C}, \tilde{\kappa}$ are

$$\tilde{\kappa} = \max\big(\kappa_{\min},\ \lceil (1+h)\,p \rceil\big), \tag{6}$$

$$\tilde{C} = \mathrm{clip}\big(\lfloor (B - \tilde{\kappa}\,b)/a \rfloor,\ C_{\min},\ C_{\max}\big), \tag{7}$$

$$\tilde{\kappa} \leftarrow \lfloor (B - \tilde{C}\,a)/b \rfloor, \tag{8}$$

where p is the burst's peak KV occupancy (blocks actually used) and h the KV headroom. Equation (6) promotes the admission floor (4) from a one-shot startup constraint to a continuously enforced invariant; (7) is the equimarginal "remainder to experts" rule for a KV step curve; and the recompute (8) reclaims any slack the cap clamp leaves, so the move never overcommits ($\tilde{C}\,a + \tilde{\kappa}\,b \le B$).

The *expert-scratch resize* grows or shrinks the per-layer GPU scratch S to $\tilde{C}$ slots, re-paging now-resident experts from W^{cpu} and dropping victims on a shrink, with π, π^{-1} and the LRU ticks resized alongside. The *KV-pool resize* reallocates the worker's per-layer KV tensors to $\tilde{\kappa}$ blocks and rebuilds the scheduler's shared block pool in place, preserving the cached `null_block` identity, done only when the pool is fully drained. To keep the transient footprint at $\max(\text{old}, \text{new}) \le B$ rather than their sum, the controller always frees the shrinking side before growing the other (on an expert grow: shrink KV, then grow scratch; on a shrink: the reverse). Every move is wrapped so a failed resize is skipped—the engine is left in its prior, usable state—and a cap dead-zone Δ suppresses moves smaller than Δ experts so the split is stable under round-to-round noise.

C Implementation Details

WiSP registers through the standard `vllm.general_plugins` entry point, so a single `pip install` suffices; the plug-in fires in every vLLM process before any model class is imported and patches three methods on the fused-MoE module—weight creation, post-load processing, and the forward. Weight creation allocates the expert masters on pinned host memory (this is what avoids the load-time GPU spike); post-load processing builds the per-layer C-slot scratch and the maps π, π^{-1}; the forward runs Algorithm 1 and dispatches the unmodified kernel with π as `expert_map`. The FP8 block-quantized path pages four tensors per expert (the weights and their block scales) and forces the Triton backend to bypass DeepGEMM's load-time post-processing, which assumes full residency. The dynamic controller's two primitives are `resize_layer_caps` (grow or shrink the expert scratch, re-paging from the host master) and a KV-pool reallocation that rebuilds the scheduler's block pool in place (Appendix B).

D Serving Design Space

This appendix expands the placement of WiSP among existing systems summarized in Section 2.3. Table 5 compares WiSP against prior autoregressive MoE-LLM serving systems along the axes that matter for our setting (routing-awareness, byte-identity, delivery, and target regime), and Table 6 lists the native serving stacks of the non-AR MoE modalities and how WiSP attaches to each.

E The Co-Activation Predictor

Section 4 uses a lightweight routing predictor to test whether expert prediction can hide page faults in low-concurrency

Table 5. AR MoE-LLM serving systems. WiSP is the low-concurrency, drop-in, reuse-caching point in the design space.

System	expert mechanism	routing-aware	byte-identical	delivery	regime
vLLM `--cpu-offload-gb` (Kwon et al., 2023)	static, layer-grained	×	✓	built-in	any (wasteful)
ktransformers (KVCache.AI, 2024)	CPU-side compute (AMX)	×	✓	engine	Intel CPU+GPU
llama.cpp (Gerganov et al., 2023)	GGUF offload	×	× (lossy)	engine	edge
MoE-Lightning (Cao et al., 2025)	pipelined paged weights	✓	✓	engine	high-batch
SiDA (Du et al., 2024)	data-aware hash offload	✓	× (approx)	engine	high-batch
FluxMoE (Liu et al., 2026) (concurrent)	stream-and-evict + planner	✓	✓	vLLM integ.	high-batch
WiSP (ours)	**cache-and-reuse via `expert_map`**	✓	✓	plug-in (entry point)	**low-concurrency**

Table 6. Serving stacks for non-AR MoE modalities and how WiSP attaches.

Modality	Representative	Native serving stack	Native expert paging	WiSP integration
MoE VLM	Kimi-VL, DeepSeek-VL2	vLLM (multimodal)	none	same engine plug-in
Hybrid SSM–MoE	Jamba	vLLM (`JambaForCausalLM`)	none; offload unsupported	same plug-in
MoE image diffusion	DiT-MoE	diffusers / repo (PyTorch)	none	∼150-line wrapper
MoE diffusion LLM	LLaDA-MoE	HF Transformers / dInfer	none	same wrapper

decode. The predictor is deliberately simple and online: it needs no gradients, calibration set, or retraining, and it uses only routing decisions that the serving engine already observes.

Construction. For each source layer ℓ and later target layer $\ell' > \ell$, the predictor keeps a co-activation table

$$A_{\ell,\ell'}(e, e'),$$

which counts how often source expert e at layer ℓ and target expert e' at layer ℓ' fire in the same forward pass. We normalize by the source marginal count to obtain an empirical conditional probability,

$$P_{\ell,\ell'}(e' \mid e) = \frac{A_{\ell,\ell'}(e, e')}{\sum_j A_{\ell,\ell'}(e, j)}.$$

At inference time, after the router has selected experts in earlier layers, the predictor scores a candidate expert e' for target layer ℓ' by summing the conditional probabilities from the experts already observed:

$$s_{\ell'}(e') = \sum_{\ell < \ell'} \gamma^{\ell' - \ell} \sum_{e \in R_\ell} P_{\ell,\ell'}(e' \mid e),$$

where $\gamma \in (0, 1]$ is a distance-decay factor. The highest-scoring experts are used as the predicted target-layer experts. The table can start empty and be learned online, or it can be initialized from a population trace or a per-user trace.

Why this is a fair prefetch test. The predictor is not intended as a new model architecture or a trained drafter. It is a low-overhead test of whether the routing signal already present during serving is useful for prefetching. It is strong enough to make the prefetch hypothesis plausible: on per-user traces it reaches nontrivial next-expert precision, around 0.5 in our Qwen3 experiments. Thus the negative result in Section 4 is not because routing is completely unpredictable. The failure mode is systems-level: even useful predictions compete with demand paging for the same PCIe bandwidth, and at tight caps the extra speculative transfers can evict experts that are still needed.

Using the same signal for memory sizing. Although the predictor does not improve single-stream decode speed, it remains useful as a memory signal. A routing trace reveals the support of the expert working set a workload repeatedly revisits. That support can be used to choose an expert cap without sweeping VRAM budgets: a global cap that is safe for all users may waste memory on most of them, while a workload-specific cap can keep the reused experts resident and return the remaining bytes to the KV cache. This is the role of routing information in WiSP: not to hide transfer latency, but to estimate the resident expert set that the allocator should budget for.

F Extended Evaluation

This appendix gives the full versions of the working-set and cross-architecture results that Section 5.4 summarizes.

F.1 The Working-Set Law, Made Measurable

Figure 4 characterizes the structure that governs where prediction *could* pay off, on a model where the working set is unusually clean to read off. A diffusion forward routes hundreds of patch tokens through the experts, so its per-forward working set is essentially the entire expert set, and the fault counts trace Denning's law exactly: below the working set the cache thrashes, and at the working set faults collapse and reuse takes over. Resident memory scales linearly with the cap, four times smaller at the tightest setting, and the output is bit-exact throughout. This is the cleanest illustration we have of *why* a single, fully-activated reference stream lets paging bound memory but not latency—and, read the other way, of why diffusion's abundant per-step compute is a regime where a prefetcher would at least have computa-

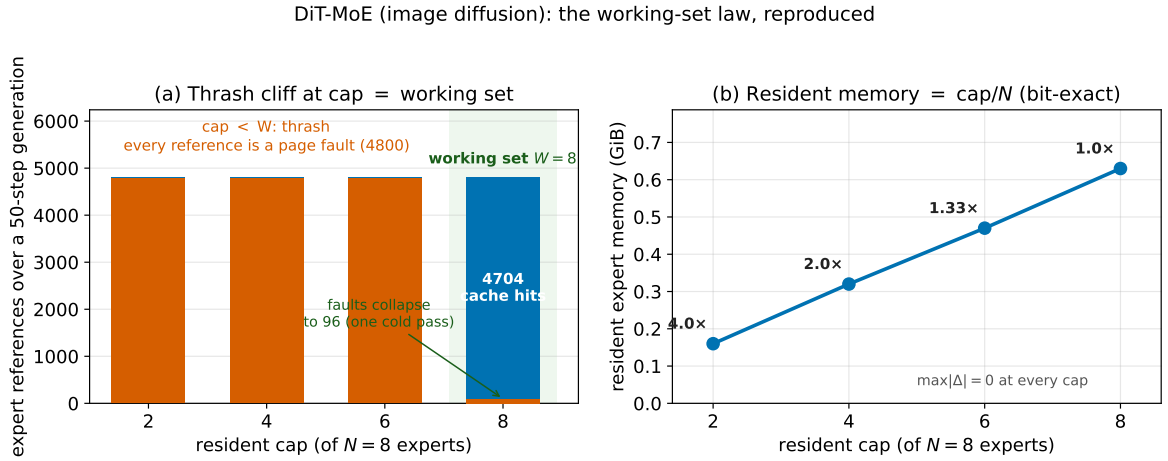


Figure 4. Working-set theory reproduced on an image-diffusion MoE (DiT-MoE-S, standalone pager). (a) Because a diffusion forward activates essentially the whole expert set over its patch tokens, a cap below the working set thrashes—every reference faults (4800 faults, 0 hits, i.e. 8 experts $\times$ 12 MoE blocks $\times$ 50 denoising steps)—and at cap = working set the faults collapse to a single cold pass (96) with 4704 hits. (b) Resident expert memory scales as cap/N, up to $4\times$ smaller, and the generation is bit-exact at every cap.

tion to hide behind, unlike single-stream LLM decode. On a fine-grained, heavily load-balanced LLM under diverse prompts (Qwen3-30B-A3B), by contrast, routing is too uniform for right-sizing to exploit and the throughput-versus-cap curve is essentially topic-independent—a conservative lower bound we report for honesty.

F.2 A Different Structure: Temporal Locality in a Diffusion LLM

DiT-MoE shows one kind of working-set structure—spatial, within a single forward, with no reuse below the working set. A diffusion *language* model exhibits the complementary kind. LLaDA-MoE generates by iteratively denoising the *same* sequence over many steps, so we ask not how a single forward routes but how routing evolves *across* denoising steps. Figure 5 makes the contrast with DiT precise. The per-step working set is large—about 96% of the 64 experts fire in every step—so a naive single-step pager thrashes just as it does on DiT, and bounding memory is again the only thing a small cap buys *within* a step. But across steps the picture changes: adjacent denoising steps agree on 44% of their per-position routing decisions, nearly twice the 23% of a shuffled-order baseline, and the agreement decays monotonically with step distance—the signature of genuine temporal locality rather than uniform reuse. DiT's structure is spatial and confined to one forward, so a prefetcher has nothing to anticipate across forwards; LLaDA's structure is temporal, so the experts a step will need are largely the ones the previous step just used, and a warm or predictive cache that survives across steps has real structure to exploit. Together these bracket the two regimes the framing predicts: a fully-activated stream with no cross-step reuse (DiT), where paging is a pure memory lever, and a fully-activated stream *with* cross-step reuse (LLaDA), where the routing signal has structure a cache or prefetcher could exploit—the diffusion counterpart of the multi-turn agent reuse we leave to future work.

G Limitations

WiSP currently runs in eager mode because the plug-in does not yet support CUDA graphs; both sides of our comparisons use eager mode for fairness. The per-layer paging decision also sits on the Python hot path. This overhead matters less in the sub-fitting regime where paging avoids large PCIe transfers, but it becomes visible as the expert cap grows and there is less to page. Moving this decision off the Python path is the most direct systems optimization. We also do not benchmark ktransformers on its Intel AMX fast path, so our comparison to that system is a regime disclosure rather than a measured loss.

Our headline iso-VRAM results for Qwen3 and Kimi-VL are measured on a physically constrained 24 GiB RTX 3090 (PCIe 4.0). This removes the emulation issue, but it also means the card cannot reach the full-footprint budget where static residency becomes faster; we characterize that crossover only on a larger GPU. The MiniMax-M2 result is likewise scoped as a capability check, not an iso-throughput win.

The MV-WSA results are also scoped. The startup-configurator result in Table 3 is a real `vllm serve` run, but it covers one KV-pressured setting. The live dual-resize controller in Table 4 is a proof of concept on real kernels: it uses the closed-form step-curve rule rather than the full

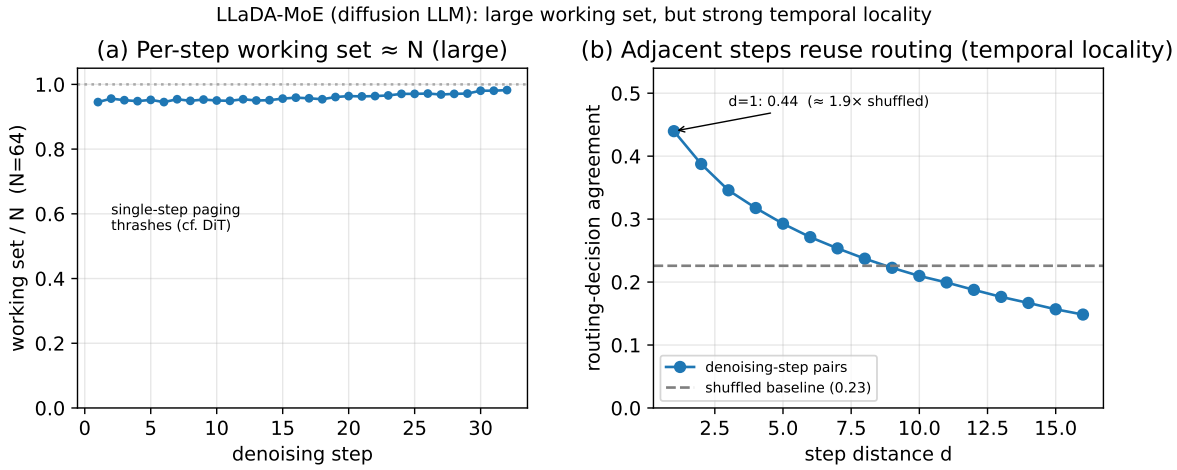


Figure 5. LLaDA-MoE has a large but temporally local working set. (a) Each denoising step activates ≈96% of all 64 experts, so—exactly as in DiT—paging a single step thrashes unless the cap is near N. (b) Yet adjacent denoising steps reuse their routing decisions: the per-position top-k agreement is 0.44 at distance one (≈1.9× a shuffled-step baseline of 0.23) and decays smoothly with step distance. The structure is temporal, not spatial.

equimarginal estimator, runs inside the in-process v1 engine rather than the multi-process `vllm serve` loop, and is evaluated on one model family and two workloads. Its byte-identity under resizing is checked by a dedicated resize-cycle test rather than a continuous regression suite. Broader workload coverage, serve-loop integration, self-calibrated cost constants, and porting the full estimator onto the live resize path are the next engineering steps.